# Online Appendix to: "TrajStory: Benchmarking Insight Discovery on Trajectory Data with Automated Agents"

Zirui Wang[1], Yuanfeng Song[2], Weicheng Wang[1], Rui Mao[3]
[1]Beijing Institute of Technology, Zhuhai [2]ByteDance, Shanghai [3]Shenzhen University, Shenzhen
{3120266674, wangweic}@bit.edu.cn, songyf@outlook.com, mao@szu.edu.cn

## I. Benchmark Details

This section provides further details of TrajStoryBench, including the source datasets, a task illustration, references for insight design, the manual data specification, insight types and difficulty levels, analytical operation counts, and data construction.

### A. Source Trajectory Datasets

We introduce the five source datasets below.

**WorldTrace.** WorldTrace [1] is a global GPS trajectory dataset introduced with UniTraj, containing approximately 2.45 million trajectories across 70 countries. Trajectory records include timestamps, latitude, and longitude, with additional map matching information. Associated metadata describe trajectory duration, distance, start and end locations, and uploaded descriptions and tags.

**Danish AIS.** Danish AIS contains vessel movement records collected through the Automatic Identification System (AIS) and released by the Danish Maritime Authority [2]. Records include timestamps, vessel identifiers, geographic coordinates, speed over ground, course over ground, heading, and navigational status. Additional fields describe vessel type, dimensions, draught, destination, and estimated arrival time.

**Porto Taxi.** Porto Taxi [3] records taxi trips from 442 taxis in Porto, Portugal. Each trip includes a trip identifier, taxi identifier, start timestamp, and a sequence of GPS coordinates sampled at 15 second intervals. The dataset also records how each trip was requested, associated caller or taxi stand identifiers where applicable, and an indicator of missing GPS observations.

**LaDe.** LaDe [4] is an industrial dataset of last mile logistics covering package pickup and delivery in five Chinese cities. It contains package and courier identifiers, service locations, area of interest information, and timestamps and positions for task acceptance and completion. Pickup records additionally specify service time windows. The release also includes courier trajectories and road networks.

**MTL Trajet.** MTL Trajet [5] is a smartphone travel survey dataset released by the City of Montréal, Canada. It contains timestamped location observations and trip records covering multiple transport modes. Trip information includes identifiers, trip geometry, and participant reported transport modes and travel purposes, linking movement traces with the context of daily travel.

### B. Task Illustration with a Benchmark Example

Each TrajStoryBench task comprises trajectory data, a high-level analytical goal, reference insights, and a narrative summary. During evaluation, agents receive only the data and goal. They must determine which attributes, comparisons, and relationships to examine, discover findings relevant to the goal, and synthesize them into a coherent summary. Figure 1 illustrates the task using an example of vessel movements. The reference insights, supporting visualizations, and summary are provided for illustration and are withheld from the evaluated agents.

### C. Studies and Reports for Insight Design

Relevant studies and domain reports inform the design of reference insights. We identify representative analytical patterns and adapt them to the attributes and mobility settings of the five source datasets. Table I lists representative references and the patterns they motivate. These sources guide the selection of relationships to investigate. Numerical values and conditions for individual tasks are established through data construction and validation, rather than copied from the source studies.

### D. Human Specification of Task Data

Once the goal and reference insights are finalized, human designers define a `pre_schema` for each task. This partial specification records the attributes, values, and relationships that require explicit control during construction. Other elements, including identifiers, timestamps, GPS points, and auxiliary statistics, can be instantiated as needed while satisfying the specified constraints and trajectory plausibility checks.

Table II summarizes the main components. The `fields` section specifies controlled attributes and optional target values, while `constraints` describes the required data composition and relationships. The specification is prepared before construction with LLM assistance and is withheld from the evaluated agents.

### E. Difficulty Levels and Insight Types

**Insight Types.** Each reference insight is assigned a type according to its primary analytical claim. *Group comparison*

TABLE I
REPRESENTATIVE STUDIES AND REPORTS USED AS REFERENCES FOR INSIGHT DESIGN.

| Dataset | References | Referenced Patterns |
|---|---|---|
| WorldTrace | [6], [7] | Studies on repeated origin–destination trips and individual route choice motivate insights on route preference, the use of alternative routes, recurring mobility regularity, and variation in travel time across repeated trips. |
| LaDe | [8], [9] | Studies on delivery operations over the final mile motivate insights on intraday service variation, spatial differences in pickup performance, route progression, and temporal patterns associated with delivery and changes in service time windows. |
| Porto Taxi | [10], [11] | Studies on taxi demand and driver behavior motivate insights on spatiotemporal demand regularity, recurring routes, movements involving taxi stands and airports, and heterogeneity across drivers and mechanisms of trip generation. |
| Danish AIS | [12], [13] | Maritime traffic studies motivate insights on traffic concentration along common routes, route deviations, vessel movement differences, and variations in speed and other operational characteristics across traffic conditions. |
| MTL-Traj | [14], [15] | Urban mobility studies motivate insights on spatial and temporal trip profiles, recurring origin–destination patterns, and relationships among trip purpose, transportation mode, timing, and broader regional mobility behavior. |

TABLE II
MAIN COMPONENTS OF THE MANUALLY DEFINED `pre_schema`.

| Component | Role | Description |
|---|---|---|
| **Domain** | *Context* | Optional context describing the trajectory domain. |
| **Goal** | *Analysis objective* | Analytical goal guiding construction. |
| **Insights** | *Target patterns* | Reference findings that the data should support. |
| **Fields** | *Controlled attributes* | Core attributes and optional target values. |
| **Constraints** | *Construction rules* | Required data composition and relationships. |
| **Prohibited fields** | *Exclusions* | Optional fields or labels to exclude. |

compares trajectory characteristics across groups or categories. *Temporal dynamics* describes changes or transitions in trajectory patterns over time. *Factor relationship* captures associations between trajectory attributes or contextual factors and trajectory outcomes. *Overall pattern* summarizes dominant, recurring, or broadly shared characteristics. *Exception/deviation* identifies observations or subsets that depart from a typical or reference pattern and serves as a complementary insight.

**Difficulty Levels.** As defined in the main paper, difficulty is determined by the number of distinct original semantic fields needed to support an insight. Quantities derived from these fields, such as speed, mean speed, variance, and route directness, do not increase the count. The four levels are *Low* (1–3 fields), *Medium* (4 fields), *High* (5 fields), and *Very High* (6 or more fields).

For example, comparing speed patterns and route directness across usage groups requires three original semantic fields: usage, trajectory coordinates, and timestamps. Speed is derived from coordinates and timestamps, and directness from trajectory geometry. The insight is therefore classified as *Low*. Difficulty and operation count describe complementary aspects of an insight: the semantic inputs it requires and the analytical steps needed to derive it.

### *F. Analytical Operation Count*

**Objective and Inputs.** We estimate the operations required by a shortest reasonable conventional analytical program that derives the requested findings from the supplied inputs. Counts describe the structure of the analysis, rather than execution time, code length, or actual tool calls. The estimator identifies the steps needed to obtain the findings without reproducing their exact numerical values. The resulting counts are estimates rather than proven global minima.

The inputs comprise the goal or question, reference insights or answers, available file schemas with field types and examples, and relevant task metadata. The protocol does not require complete raw tables, reference code, or image pixels. Reference `evidence`, `description`, and `insight_value` fields clarify the target findings but do not constitute source data or available input columns. Explicitly requested statistics are included; background totals, explanatory diagnostics, and summary prose

TABLE III
ANALYTICAL OPERATORS AND COUNTING RULES. OPERATIONS ARE COUNTED WHEN NEEDED TO DERIVE THE TARGET FINDING.

| Operator | When Counted | Merging Rule |
|---|---|---|
| SELECT | Retrieve an existing fact required as a final output. | Reading files, matching keys, and formatting an option letter do not add operations. |
| FILTER | Form a necessary subset of the input records. | Conditions defining the same subset count once. Subsets with different semantic roles and filters that depend on computed values remain separate. |
| ORGANIZE | Perform necessary grouping or chronological ordering. | Grouping by several keys counts once. Display sorting is excluded; ranking and extremum judgments use COMPARE. |
| COMPUTE | Derive, aggregate, predict, or classify one type of measure at one analytical stage. | The same measure across objects or groups counts once. Distinct measures and dependent stages count separately; internal arithmetic and model training are not expanded. |
| COMPARE | Compare, rank, or judge extrema or trends for one measure. | Comparing one measure across groups counts once; different measures count separately. Computing group statistics does not replace an explicitly required comparison. |

do not automatically create additional targets. TrajStoryBench uses the schemas of `input_*.csv` files, excluding derived output tables.

**Operators and Granularity.** Table III defines the five operators and their merging rules. Each COMPUTE derives one type of measure at one analytical stage. Computing that measure across several objects, groups, or files counts once within a program. Distinct measures and successive stages with dependencies are counted separately. Prediction and classification also use COMPUTE. When a finding requires comparison, COMPARE remains necessary after group statistics have been calculated. Ranking or selecting an extremum does not incur an additional sorting operation. File loading, joins and key matching, schema inspection, code execution and repair, plotting, formatting, prose generation, and recommendations are excluded.

**Counting Examples.** If speed is already supplied, calculating its mean requires one COMPUTE. Given dates and individual trip distances, comparing mean distances between weekdays and weekends requires three operations: ORGANIZE groups, COMPUTE group means, and COMPARE. Deriving trip distances from trajectory points adds one COMPUTE. The distances between adjacent points and their sum form a single trajectory length calculation; calculating group means is a separate stage.

When speed and an anomaly threshold are both absent, detecting speed anomalies from coordinates and timestamps requires three operations: COMPUTE speed, COMPUTE a threshold using the conventional $1.5 \times \mathrm{IQR}$ rule, and FILTER anomalies. Supplied quantities are used directly, and any method specified by the task takes precedence. An additional operation to count anomalies is required only when their number is an explicit target. Predicting the next location requires one COMPUTE, with necessary input filtering counted separately. Predicting a destination and retrieving nearby POIs requires COMPUTE followed by SELECT, provided the necessary mapping is available.

**Task and Insight Levels.** For discovery tasks, the estimator produces one program covering all required reference insights and a separate program for each individual insight. Within the program for the entire task, necessary operations may be shared only when their inputs, subset, measure, and analytical stage are identical. Using the same data does not justify merging different computations or comparisons. Each individual insight is evaluated from the supplied inputs, without assuming that another insight has completed any prerequisites. The task count therefore comes directly from its shared program. QA is evaluated per question. HeurekaBench is reported at a single level because each evaluated question has one complete reference answer.

**Estimation and Aggregation.** GPT-6 Luna [16] estimates each task, QA question, and individual insight three times, with temperature set to 0 and reasoning disabled. Its structured response contains a status, an ordered list of operators with concise descriptions, and any assumptions that affect the count or missing context. The count is the length of the operation list. For each benchmark and level, we divide the sum of counts across valid trials by the number of valid trials. Valid trials are retained even when another trial for the same unit fails.

API failures, parsing failures, and unscorable cases are excluded from the mean and are never assigned zero. A case is unscorable only when the target cannot be interpreted or essential information is missing, preventing a defensible analytical program. Missing exact values, model names, or parameters alone do not justify exclusion. The estimator instead follows a fixed precedent or a conventional method and states its assumptions. Retries use the same inputs and settings, preserve the original failures, and replace the corresponding trials without increasing the denominator. The reported counts remain model estimates pending a full semantic audit; successful responses and agreement across trials do not establish semantic correctness.

**Evaluation Scope and Results.** The following paragraphs describe the evaluated samples and their mean operation counts. For discovery benchmarks with multiple reference insights, paired values denote the task and insight means, respectively. Sample sizes describe the units evaluated here and may differ from the overall benchmark sizes in the main paper. Each unit has three planned trials, and valid trials are retained as described above.

- **TrajStoryBench.** We evaluate all 125 tasks and 423 reference insights, obtaining mean counts of 12.35 / 4.65. The estimates use 375/375 valid trials for tasks and 1261/1269 for insights. Input schemas are taken from the original `input_*.csv` files.
- **InsightBench.** The evaluated sample contains 88 tasks and 354 insights. Mean counts are 11.79 / 3.66, based on 263/264 valid task trials and 1015/1062 valid insight trials.
- **InsightEval.** We evaluate 100 tasks and 940 insights. Mean counts are 17.94 / 3.84, based on 299/300 valid task trials and 2786/2820 valid insight trials. Both levels use the same aggregation rule as the other benchmarks.
- **UniDataBench.** We use five available example tasks and their 24 reference insights. Mean counts are 9.20 / 3.85, based on 15/15 valid task trials and 72/72 valid insight trials. These estimates characterize the available examples rather than the full benchmark.
- **HeurekaBench.** We evaluate 50 open questions, each with one complete reference answer treated as a single insight. Although these questions are grouped with discovery benchmarks, their task and insight units coincide: deriving the complete answer also derives the only reference insight. The mean count is therefore reported once as 4.01, based on 145/150 valid trials. Writing 4.01 / 4.01 would repeat the same estimate rather than represent two separately evaluated levels.
- **UrbanDS-Bench.** We proportionally sample 300 questions by category and subtype. The mean count is 4.37, based on 893/900 valid trials. Sampling covers the available urban analysis categories rather than only trajectory questions.
- **MobQA.** The original sample contains 300 questions across 34 subtypes: 58 factual retrieval, 131 free form, and 111 multiple choice questions. Its mean count is 1.78, based on 900/900 valid trials. Inputs contain the default day, time, and coordinate fields, without optional annotations for transportation mode, POIs, or travel reasons. Inferring these attributes uses `COMPUTE`; expressing a result as a choice letter does not add `SELECT`.
- **TravelBehaviorQA.** We use all 10 available public examples. The 30/30 valid trials give a mean count of 3.83. The result describes these examples rather than the full benchmark.
- **USTBench.** We sample 300 questions using strata defined by domain, reasoning dimension, and applicable subtypes. The mean count is 2.07, based on 900/900 valid trials.
- **POI-QA.** We use 300 labeled English validation questions for POI name generation. The mean count is 2.00, based on 900/900 valid trials. This sample represents that task type rather than every POI task in the benchmark. Any required prediction is counted as `COMPUTE` within the same five operator taxonomy.

Proportional quotas use the largest remainder method, followed by sampling without replacement within strata using seed 20261021. For a stability check, another 300 MobQA questions were sampled using the original category quotas and seed 20261022, with no overlap with the original sample. Their mean count was approximately 1.80, compared with 1.78 for the original sample. The main comparison retains the original sample and denominator.

## II. Benchmark Quality Assessment

We assess the quality of benchmark data and reference insights using a common rubric for human review and automated assessment. This assessment concerns the benchmark materials themselves; agent performance is evaluated separately in the main paper.

### A. Assessment Criteria

Following the four dimensions of UniDataBench [17], each evaluation unit receives an integer score from 1 to 5 for each dimension. *Data suitability* concerns whether the fields, coverage, granularity, and record quality support the intended analysis. *Relevance* concerns whether the reference insights address the analytical goal. *Consistency* concerns compatibility among the insights and their supporting evidence. *Rationality* concerns whether the conclusions are plausible and supported by the available data, figures, and reasoning. An asserted reference value or a plausible explanation alone does not establish evidential support.

### B. Automated Assessment

GPT-6 Luna [16] assesses each unit in two independent rounds, with temperature 0 and reasoning disabled. Both rounds use the same frozen inputs and rubric. The judge returns a score and a short justification for each dimension, together with problematic insights and limitations. We first average the two scores for each unit and then average across units, assigning equal weight to every unit within a benchmark.

The evaluated sets contain 125 TrajStoryBench tasks, 100 InsightBench cases, 100 InsightEval cases, and 12 HeurekaBench source papers. All 337 units have two successful judgments, yielding 674 judgments after retries. These are the samples used for quality assessment and need not coincide with the samples used for operation counting. In particular, HeurekaBench is assessed per source paper here, rather than per question.

The judge receives the goal, reference claims, and available evidence, including data profiles, example records, and figures where supplied. Evidence availability differs across benchmarks. For HeurekaBench, the H5AD profiles expose structural metadata, such as matrix shapes and field names, rather than expression values; the input contains no reference figure or analysis code. Scores therefore characterize support in the

supplied materials and do not establish that every reference result has been independently reproduced.

### C. Human Review and Matched Comparison

The human assessment contains 70 scored units: 20 each from TrajStoryBench, InsightBench, and InsightEval, and 10 source papers from HeurekaBench. A human reviewer uses the same four dimensions and the 1–5 scale. The review interface presents goals, reference insights, source material links, and available figures, without displaying AI scores. It also provides optional fields for recording evidence checks and limitations. The sample roster is fixed; it combines retained cases and deterministic selection based on hashes rather than a newly drawn random sample.

For comparison with human ratings, we restrict the AI results to these same 70 units and retain the two-round average for each unit. The resulting means use identical units and weights within each benchmark. The larger automated sets are reported separately. Because the recorded human assessment supplies one rating per unit, we do not estimate agreement between multiple human reviewers.

### D. Quality Assessment Results

Figure 2(a)–(d) reports automated ratings across the evaluated sets. TrajStoryBench obtains mean scores of 3.748 for data suitability, 4.912 for relevance, 4.312 for consistency, and 3.464 for rationality, achieving the highest means among the four evaluated sets in every dimension. Its relevance score is close to that of HeurekaBench (4.912 versus 4.875). These comparisons involve different domains, evaluation units, and available evidence; the descriptive means do not establish statistical significance.

Figure 2(e)–(h) compares ratings on matched units. For the 20 TrajStoryBench tasks, human means are 3.750, 4.800, 4.150, and 3.500 in the same dimension order, compared with AI means of 3.825, 4.925, 4.400, and 3.775. Both assessments rank TrajStoryBench highest across the four benchmarks in every dimension. AI means exceed human means by 0.075–0.275 points on this subset. Across all four benchmarks and dimensions, absolute differences between paired aggregate means range from 0 to 0.300 points. Similar aggregate means do not imply agreement on individual cases.

The results provide complementary human and AI evidence for the comparative quality of TrajStoryBench, including its rationality. This pattern holds across all four dimensions on the matched samples. However, a single reviewer, fixed samples, and differences in domains and available evidence limit the generality of the comparison.ferences in domains and available evidence limit the generality of the comparison.

## III. Summary Evaluation

We evaluate the final summaries against the authored reference summary for each of the 125 tasks. Table IV reports summary G-Eval for both backbone models. All summaries are judged by GPT-6 Luna in two independent rounds with temperature 0 and reasoning disabled. We average the two scores for each task and then average equally across all 125 tasks; missing or empty summaries receive zero. Evaluation uses the saved summaries and references without regenerating outputs.

TABLE IV
Summary G-Eval on TrajStoryBench tasks.

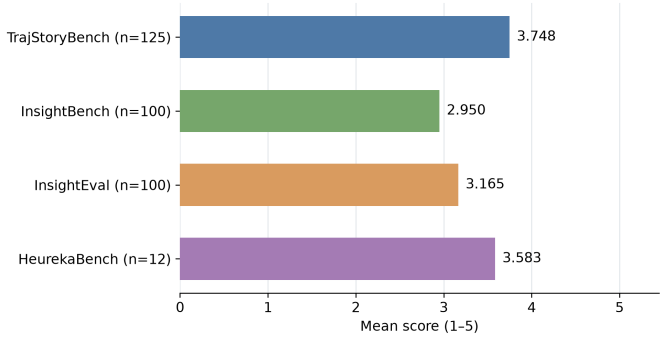
| Agent | GPT-6 Luna | Qwen3.8-Flash |
|---|---|---|
| LLM-only [16], [18] | 0.1632 | 0.3232 |
| AgentOrchestra [19] | 0.2416 | 0.3212 |
| AgentPoirot [20] | 0.3880 | 0.2988 |
| CodeGen [21] | 0.2184 | 0.3504 |
| Pandas Agent [22] | 0.3360 | 0.3668 |
| DSAEval [23] | 0.3068 | 0.3248 |
| TrajStoryAgent (Ours) | **0.4188** | **0.3948** |

TrajStoryAgent achieves the highest summary G-Eval with both backbones, scoring 0.4188 with GPT-6 Luna and 0.3948 with Qwen3.8-Flash. The strongest corresponding baselines are AgentPoirot (0.3880) and Pandas Agent (0.3668), giving absolute improvements of 0.0308 and 0.0280. These correspond to relative gains of approximately 7.9% and 7.6%, respectively.

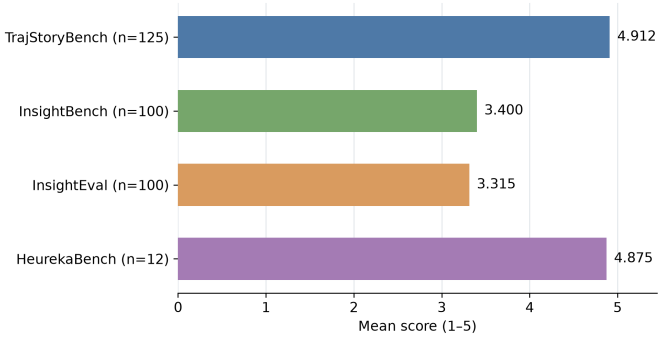
Compared with LLM-only, TrajStoryAgent improves summary G-Eval by 0.2556 with GPT-6 Luna and 0.0716 with Qwen3.8-Flash. CodeGen obtains 0.2184 and 0.3504, respectively, also remaining below TrajStoryAgent under both backbones. Although baseline rankings vary across backbones, TrajStoryAgent ranks first in both settings. By evaluating the final narrative as a whole, these results complement the insight-level comparisons in the main paper and show that TrajStoryAgent's advantage also extends to summary quality under the shared judging protocol.



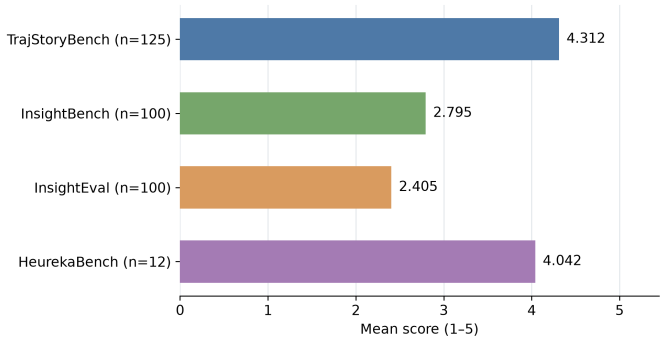

Fig. 1. Illustration of a TrajStoryBench task. Agents receive trajectory data and an analytical goal. The reference insights, supporting visualizations, and narrative summary show the intended findings and are withheld during agent evaluation.



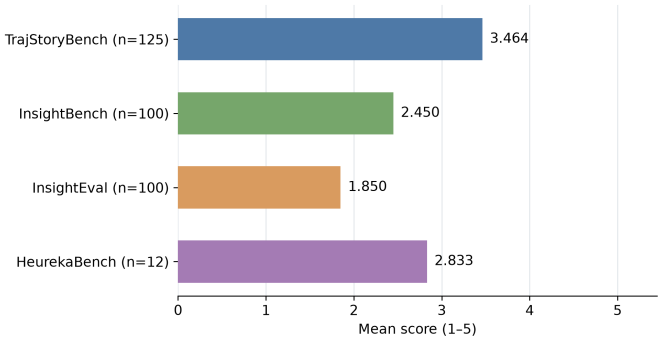

(a) Data suitability.



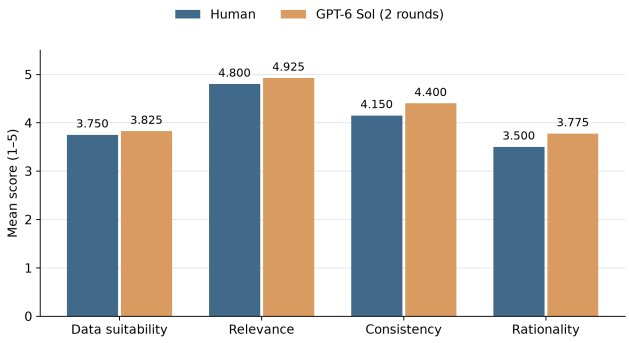

(b) Relevance.



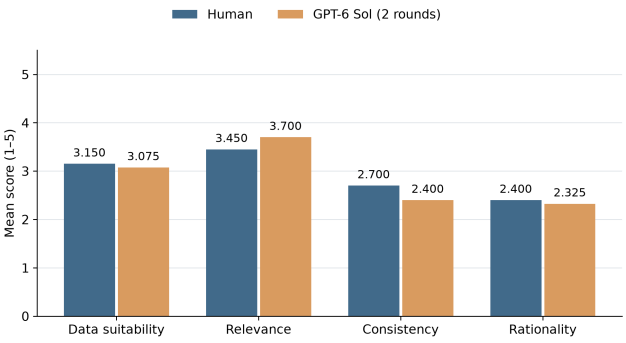

(c) Consistency.



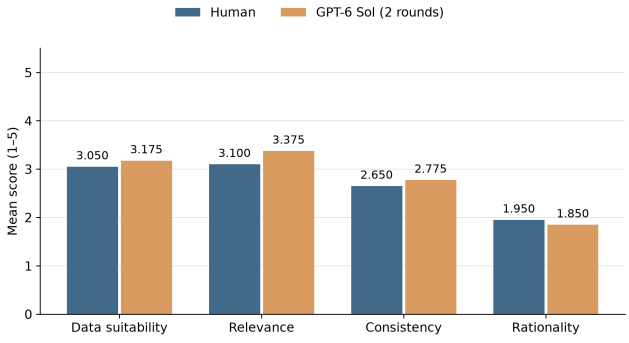

(d) Rationality.



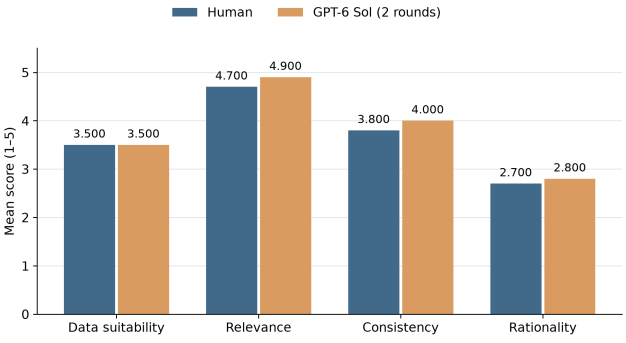

(e) TrajStoryBench.




(f) InsightBench.




(g) InsightEval.




(h) HeurekaBench.

Fig. 2. Benchmark quality assessment on a 1–5 scale. (a)–(d) GPT-6 Sol ratings for 125 TrajStoryBench tasks, 100 InsightBench cases, 100 InsightEval cases, and 12 HeurekaBench papers. (e)–(h) Human and AI ratings on the same 20, 20, 20, and 10 units, respectively. AI scores average two independent rounds per unit, with temperature 0 and reasoning effort set to none. All means assign equal weight to units within each benchmark.

## IV. Prompts

We provide the principal prompt templates for the evaluated TrajStoryAgent v5 workflow and the benchmark assessments. Angle brackets denote supplied data or runtime context, such as the goal, table profile, computed evidence, and error messages. Instruction wording is retained, with line wrapping adjusted for readability. The shared input policy is displayed once rather than repeated before each workflow prompt. The analysis tool API is represented by a placeholder; it specifies the available loading, computation, figure, and result emission methods. Format retries additionally supply the previous response and validation error.

### *A. TrajStoryAgent*

The templates follow the evaluated configuration with Memory Manager, relation profiling, and question coverage review enabled. The verification prompt is followed by the disagreement review instructions; a branch position update is used only for admitted disagreement topics. Observation export supplies the deterministic conditional decomposition and compression procedures. These procedures operate on computed observations; the language model subsequently interprets their certificate facts. If refinement is unsuitable, the original insight is retained.

**Prompt 1: Shared input policy**

```
INPUT POLICY:
Only the named projected input tables and visible columns are available.
Previously supplied derived columns may be absent by design. Compute needed
derived quantities from visible fields where possible; do not recover them
from original CSVs, unlisted files, reference insights or prior experiment outputs.
If visible inputs cannot support a quantity or conclusion, state that limitation.
```

**Prompt 2: Data profiling**

```
SYSTEM:
Jointly infer column semantics and concise structural relationships from deterministic table profiles
and complete sample rows. Return JSON only.

USER:
DATA CONTEXT:
<DATA_CONTEXT>

Jointly classify every column using its name, raw dtype, statistics, per-column values,
and the supplied complete rows. Complete rows preserve relationships among values in the
same record. The random rows are a deterministic random sample (seed 0), not additional
evidence for answering the analytical goal.

For semantic_role use exactly one of:
['attribute', 'boolean', 'category', 'direction', 'distance', 'duration', 'geometry', 'identifier',
'latitude', 'longitude', 'measure', 'sequence', 'speed', 'timestamp']

Also propose only useful structural relationships inside each individual table. The
relationship label is free-form: do not force it into identifier/temporal/spatial/semantic.

For cross-table analysis, describe the relationship between each relevant pair of tables.
Relationship labels remain free-form. State what the tables mean to each other, not
how to join them: omit join keys, candidate IDs, cardinalities, overlap scores, and
code instructions from relationship labels and descriptions. Inspect
deterministic_cross_table_join_candidates and select candidate IDs only
when the fields are a semantically valid row-linkage key without incorrect row
multiplication. Reject coincidental overlaps in categories, timestamps, measures,
or coordinates. Candidate IDs are internal A0 clues and are not shared with
later stages. An empty list is correct when the tables are related conceptually
but have no safe join candidate.

Do not answer the analytical goal and do not propose insights. Use no columns or files
absent from the supplied profile.

Return exactly:
{
  "column_semantics": [
    {"file_id":"F1","columns":[
      {"name":"column_name","semantic_role":"attribute","description":"what the field represents",
        "unit":null}
    ]}
  ],
  "intra_file_relations": [
    {"file_id":"F1","relationship":"free-form relationship label",
      "fields":["field1","field2"],"description":"..."}
  ],
  "cross_file_relations": [
```

```
    {"files":["F1","F2"],"relationship":"free-form table relationship label",
      "description":"how the two tables relate as tables",
      "join_candidate_ids":["J1"]}
  ]
}

PROFILE AND COMPLETE ROWS:
<PROFILE_TEXT>
```

**Prompt 3: Initial question planning**

```
SYSTEM:
You are TrajStory A1. Plan initial questions only; do not analyze data or propose follow-ups. Return
JSON only.

USER:
HIGH-LEVEL GOAL:
<GOAL>

DATA CONTEXT:
<DATA_CONTEXT>

TRAJECTORY DATA PROFILE:
<DATA_PROFILE_JSON>

Generate exactly three complementary sub-questions that directly serve the goal.
Do not map them mechanically to fixed temporal/spatial/mode categories. Do not ask a
question merely because a field exists. Each sub-question must pursue one clear analytical
objective, not necessarily one metric. Multiple related metrics and joint grouping conditions
are allowed when they serve that same comparison, pattern, or relationship. For example,
comparing vessel size across route classes may jointly examine length, width, and tonnage;
comparing movement by transport mode within each trip purpose is also one objective.
Do not split one coherent comparison into separate questions merely because it uses several
metrics. Select relevant metrics rather than listing every available field. Write one
interrogative sentence with a single question mark. Grouping and time-window conditions
may define the analysis but must not introduce an independent analytical objective. Do not combine
requests such as describing
a pattern AND locating anomalies, comparing groups AND explaining causes, or measuring
an association AND testing a separate outcome. Distribute distinct analysis goals across
the three sub-questions; leave three additional goal-relevant directions, connections, or
refinements for adaptive follow-ups. Do not
perform statistical inference. For each question, plan its base analysis unit and optional
aggregation unit. This plan will be fixed before the separate code generator runs.
When a question needs fields from multiple tables, use the profiled table
relationship to identify a plausible analysis direction. The code stage must
check any chosen linkage key and cardinality against the data.

Return:
{"sub_questions":[
{"question":"...","analysis_granularity":{"base":"point|trajectory/trip|user/entity|group","aggregation":"
   field/grouping
or null"}},
  {"question":"...","analysis_granularity":{"base":"...","aggregation":null}},
  {"question":"...","analysis_granularity":{"base":"...","aggregation":"..."}}
]}
```

**Prompt 4: Analysis code generation**

```
SYSTEM:
Generate the analysis program. Output Python source only; never output JSON, analysis prose, or
interpreted conclusions.

USER:
HIGH-LEVEL GOAL:
<GOAL>

QUESTION (must remain unchanged):
<QUESTION>

FIXED ANALYSIS GRANULARITY PLANNED BY THE QUESTION AGENT:
<ANALYSIS_GRANULARITY_JSON>

PARENT ANALYSIS CONTEXT (for follow-up questions):
<PARENT_CONTEXT_JSON>
<ERROR_CONTEXT>

A2 BRANCH TYPE:
<BRANCH_KIND>

SIMPLE-FIRST ANALYSIS CONTRACT:
<INITIAL_OR_FOLLOW_UP_CONTRACT>

NAMED INPUT FILES:
<NAMED_INPUT_FILES_JSON>

TRAJECTORY DATA PROFILE:
<DATA_PROFILE_JSON>
<REVISION>
Generate Python that loads the full input files and computes the results needed to answer
the question at the fixed analysis granularity. Do not use profile samples as evidence.

STABLE ANALYSIS TOOL API:
<ANALYSIS_TOOL_API_REFERENCE>

The code must:
- perform the analysis rather than hard-code results;
- answer the fixed question's objective using all related metrics it requests; do not
  narrow a multi-metric comparison to one convenient field or add adjacent hypotheses;
- load only listed local inputs through AnalysisSession.load_table(), never direct paths;
- when combining tables, inspect the relevant fields and relationship, validate
  the chosen key and cardinality, and avoid unintended many-to-many expansion;
- prefer the supplied trajectory/statistics helpers over reimplementing routine operations;
- use the simplest sufficient method: counts/proportions, grouped summaries, direct
  ordering or geometry checks before considering statistical models;
- create one or two distinct decision-relevant visualizations through
  session.save_figure(); use separate filenames and figures when saving two;
  the session automatically includes every saved figure in emit_result(), so do
  not reconstruct their metadata from the images;
- choose the visualization type to match the question and attach its actual plotted data
  as the required structured observation: chart_type, x_axis, y_axis, and statistic_names;
  use at most 50 representative axis values and preserve multiple plotted series as a
  name-to-values mapping rather than collapsing them into a prose description;
- emit every decisive value encoded by the plot in the bounded result table or statistics
  as well, so the structured visual evidence remains independently auditable;
- emit at most one bounded result table and a small set of decisive statistics;
- finish with session.emit_result(), which writes the raw result and scope automatically;
```

```
- put computed values, including null/negative results, into statistics or tables;
- never write an insight, answer, justification, narrative, or interpretation;
- target 60--140 lines and avoid duplicate calculations or alternate methods.

Return Python source only. Do not return JSON and do not explain the code. A Markdown
```python fence is accepted, but raw Python is preferred.

INITIAL CONTRACT:
Address one clear descriptive or comparative objective using all related metrics requested by the
question. Multiple metrics and joint grouping conditions are allowed within that objective; combine
them in the bounded result table rather than dropping metrics or inventing additional objectives. Do
not add robustness checks, causal controls, alternate baselines, clustering, or predictive models
unless the fixed question explicitly requires one. Leave other goal-relevant directions, connections,
or refinements to follow-up branches.

FOLLOW-UP CONTRACT:
Answer the selected next question directly, whether it broadens coverage, connects existing evidence,
or refines a prior result. Include all related metrics it requests. Multiple metrics and joint grouping
conditions can serve the same objective. Reuse the parent computation only where relevant and do not
introduce a second analysis objective or multiple competing baselines unless the question explicitly
requests a method comparison. Follow parent agreement_context.use_rules: deferred insights are
hypotheses to check, not premises. Preserve method/scope conditions and never mix incompatible
denominators.
```

**Prompt 5: Analysis code repair**

```
SYSTEM:
You are TrajStory's Python code repair worker. Diagnose and repair the supplied analysis program.
Output the complete repaired Python source only; never output JSON or interpretation.

USER:
QUESTION (unchanged):
<QUESTION>

FIXED ANALYSIS GRANULARITY:
<ANALYSIS_GRANULARITY_JSON>

PARENT ANALYSIS CONTEXT (for follow-up questions):
<PARENT_CONTEXT_JSON>
<ERROR_CONTEXT>

A2 BRANCH TYPE AND SIMPLE-FIRST CONTRACT:
<BRANCH_KIND>
<INITIAL_OR_FOLLOW_UP_CONTRACT>

STABLE ANALYSIS TOOL API:
<ANALYSIS_TOOL_API_REFERENCE>

VISUALIZATION CONTRACT FOR THIS ATTEMPT:
Preserve one primary analysis, at most one table and one or two distinct structured visualizations; use
unique filenames and repair missing chart_type/x_axis/y_axis/statistic_names metadata. Remove unrelated
or duplicate analysis while repairing.

NAMED INPUT FILES:
<NAMED_INPUT_FILES_JSON>

EXECUTION OR VALIDATION FAILURE:
<FAILURE>
```
```

```
FULL FAILURE HISTORY (oldest to newest):
<FAILURE_HISTORY_JSON>

Repeated failures mean an earlier repair was ineffective. Use this history to identify
the root cause, then inspect every occurrence of the implicated field, helper, shape, or
metadata pattern in the complete previous program before producing the next revision.

PREVIOUS CODE:
<PREVIOUS_CODE>

Repair only the Python source. Preserve the question, analysis granularity, and intended calculation.
The repaired
code must load approved inputs through AnalysisSession.load_table() and finish with
session.emit_result(). Supporting scope is derived automatically and must not be manually
constructed. Preserve one primary analysis and at most one table. It must not produce a
insight or justification.
For cross-table operations, check the chosen key and cardinality against the
available tables before merging; do not infer a key solely from similar names.

Return Python source only. Do not wrap it in JSON and do not explain it.

METADATA-ONLY FALLBACK (replaces the visualization contract when applicable):
This is the final metadata-only fallback after repeated visualization failures. Do not call plotting
helpers or session.save_figure(). Remove figure creation and save calls while preserving the original
data calculations and question. Put the decisive computed values formerly represented by the chart,
including all metrics requested by the question, into statistics or one bounded result table. Do not
infer values from the failed image or error text. Use session.emit_result() with computed evidence.
This attempt alone is allowed to omit the visualization; supporting_scope still comes from
AnalysisSession.load_table().
```

**Prompt 6: Interpretation of computed results**

```
SYSTEM:
You are TrajStory's post-execution result interpreter. Interpret computed evidence; never generate or
repair code. Return JSON only.

USER:
HIGH-LEVEL GOAL:
<GOAL>

QUESTION:
<QUESTION>

ANALYSIS GRANULARITY:
<ANALYSIS_GRANULARITY_JSON>

COMPUTED RAW RESULT:
<COMPUTED_RESULT_JSON>

PARENT AGREEMENT CONDITIONS (if any):
<AGREEMENT_CONTEXT_JSON>
Respect the use_rules and distinguish deferred hypotheses from computed findings.
If the current result resolves or contradicts a prior argument, state its argument ID,
the matching scope, and the new computed basis in justification. Otherwise preserve
uncertainty. Parent agreement does not supply new measured facts.

Interpret only the supplied computed result. Treat each visualization.observation as
structured visual evidence: inspect its chart_type, x_axis values, y_axis values/series,
and referenced statistics to identify decision-relevant patterns, contrasts, extrema, or
```

```
exceptions. Do not infer from the image path or prose description alone. Do not write
Python, request another analysis, invent values, or use facts absent from the result.
State a negative/null result honestly. The justification must point to specific supplied
statistics, table values, or structured x/y visual-observation values.
Limitations must describe only material constraints visible in the result.
<OPTIONAL_VISUALIZATION_FAILURE_INSTRUCTION>

Return exactly:
{"insight":"one coherent factual insight, at most 180 words","justification":"evidence-linked
explanation",
  "limitations":["..."],"evidence_strength":"strong|moderate|weak"}

WHEN THE VISUALIZATION FAILED (otherwise omitted):
The chart failed after the code repair budget. There is no structured visual observation. Base the
conclusion only on computed statistics and result-table values, and acknowledge the missing
visualization.
```

**Prompt 7: Evidence relevance verification**

```
SYSTEM:
You are TrajStory A3. Check question relevance and high-level goal relevance. Also summarize material
cross-evidence disagreements using the supplied schema. Do not settle disputed facts or change computed
results. Return JSON only.

USER:
HIGH-LEVEL GOAL:
<GOAL>

DATA PROFILE:
<DATA_PROFILE_JSON>

QUESTION/EVIDENCE PACKAGES:
<QUESTION_EVIDENCE_PACKAGES_JSON>

For each branch return VALID if the evidence directly answers its question and still
serves the high-level goal. Otherwise return REVISE with concrete issues, a revision
target, and an instruction. Use revision_target QUESTION when the question itself is
off-goal, duplicates an answered objective, requests unavailable fields/relations, or
cannot be fully answered from the supplied profile. Use EVIDENCE only when the question
is appropriate and answerable but the generated evidence fails to answer it. Do not ask
for evidence regeneration when changing the question is necessary.
For VALID, revision_target and revision_instruction are unused: omit them or use null.
For REVISE, both an explicit EVIDENCE/QUESTION target and a nonempty instruction are required.

Return:
{"results":[{"question_id":"Q1","status":"VALID|REVISE",
"issues":[],"revision_target":"EVIDENCE|QUESTION","revision_instruction":""}]}

<AGREEMENT_REVIEW_INSTRUCTIONS>
EXECUTED EVIDENCE AND CODE:
<EXECUTED_EVIDENCE_AND_CODE_JSON>
```

**Prompt 8: Cross-branch disagreement review**

```
In the SAME response, also return "agreement_topics": [] if no material cross-branch
disagreement exists. Do not invent conflict between different questions, populations,
units, time ranges, or explicitly conditional methods. Missing discussion is not rejection.
Inspect the executed code as well as evidence. Identify only disagreements affecting
subsequent use of the evidence. Each topic has 2 or 3 participating evidence IDs and
2--8 atomic arguments (bounded insights or method-use proposals already grounded in evidence).
Include unaffected parts only when needed to express partial agreement within that topic.
Record exact scope/method conditions in statements. Do not declare a method correct simply
because it ran, was used first, or has more supporters. Code problems require concrete citations.
Each argument has source evidence IDs, a nonempty basis pointing to supplied statistics or code,
and IDs of arguments it requires. Attacks are pairs of incompatible arguments in this topic,
not differences of scope. Arguments and dependencies are local to each topic, with no cycles.
No new calculations or fabricated numbers. Do not request code repair merely for disagreement;
REVISE remains reserved for question/goal irrelevance.
Schema in addition to results:
"agreement_topics": [{"topic_id":"T1","description":"...",
"evidence_ids":["E1","E2"],"arguments":[
{"argument_id":"a","statement":"...","evidence_ids":["E1"],
"basis":"statistic name or code excerpt and its relevance","requires":[]},
{"argument_id":"b","statement":"...","evidence_ids":["E2"],
"basis":"...","requires":[]}],"attacks":[["a","b"]]}]
At most 3 topics. Return all required keys, including empty lists.
For each argument add result_ref={"evidence_id":"E1","statistic_index":0}
when it is an exact assertion about a supplied executed scalar. Otherwise use null.
statistic_index is zero-based. Do not invent references, comparison_records, scope,
units, or method metadata. The program admits only incompatible scalar results
with matching execution-recorded contexts; qualitative/custom/legacy assertions
remain unconfirmed when this context is absent. Candidate prose is never proof of conflict.
```

**Prompt 9: Branch position update**

```
SYSTEM:
You are the original TrajStory analysis branch updating your position once after reading shared
evidence. Do not defend your original answer automatically. For EVERY argument in EVERY supplied topic,
return accept, reject, or unknown, with a concrete evidence/code reason. Unknown means not established,
not rejection. Accept only the exact bounded statement, including method/scope conditions. Do not
compute, change measured values, rewrite code, or invent evidence. An alleged error without sufficient
grounds must remain unknown. Return JSON:
{"positions":[{"topic_id":"T1","arguments":[{"argument_id":"a","stance":"accept|reject|unknown","reason
   ":"..."}]}]}.

USER (JSON):
{
  "goal": "<GOAL>",
  "your_evidence_id": "<EVIDENCE_ID>",
  "topics": "<SELECTED_TOPICS>",
  "shared_evidence": "<PARTICIPATING_EVIDENCE_AND_CODE>"
}
```

**Prompt 10: Memory Manager**

```
SYSTEM:
You are TrajStory's dedicated Memory Manager Agent. Your sole job is to
compress completed, verified first-round work into compact, faithful memory.
Question and Finding go to the next-question generator; Errors appear in coding
go only to the follow-up code writer. You are not the question generator, code
writer, verifier, or evaluator.

Return exactly three fields: Question, Finding, and Errors appear in coding.
Record every initial question, including unsuccessful ones, so it is not
repeated. Record only supplied validated findings, retaining evidence IDs,
their originating Q-ids, qualitative conclusions, necessary population/time/method
conditions, limitations, and conflict-use restrictions. In Finding, do not
include measured result values such as counts, percentages, averages,
distances, durations, sample sizes, or ranges, whether written in digits or
words. The original evidence holds those details. A brief scope distinction
such as the number of compared route classes is allowed only if needed to
avoid misgeneralization. Labels such as day_1, Day 1, and route_2 identify
categories and may be retained; they are not measured result values. Keep a numeric condition in
Question only if
essential to identify its objective.
The source insights may show [value omitted] or [quantity omitted] where a
measured value was removed before you saw it. Never copy those placeholders
into memory; state only the qualitative pattern they support.

Compress for planning, not archival: use one terse entry per question or
finding ID, not a copied paragraph. Aim for at most 260 characters per Question
and 450 per Finding; these are brevity targets, not reasons to omit a necessary
restriction. Keep the analytical
objective in Question. In Finding, use a brief qualitative result and only
the scope or caveat needed to avoid repeating or misusing it. Omit repeated
goal/profile text, raw code, statistics, method narration, and generic
commentary. Preserve substantive direction and restrictions while shortening
wording. Aim for about 2,000 characters overall in an ordinary three-question
case. Hard limits are 400 characters per Question, 900 per Finding, and 4,000
for the whole JSON; the measured-result ban is never relaxed.

Never turn a hypothesis, failed analysis, or unreviewed disagreement into a
fact. Summarize each supplied exhausted coding failure once under its Q-id so
the follow-up code writer can avoid it; omit transient or resolved retries.
Never invent data, fields, joins, or results.
Do not create second-round questions. Return JSON only.

USER:
HIGH-LEVEL GOAL:
<GOAL>

FIRST-ROUND QUESTIONS (including failed branches):
<FIRST_ROUND_QUESTIONS_JSON>

VERIFIED FINDINGS ONLY:
<VERIFIED_FINDINGS_JSON>

EXHAUSTED CODING FAILURES:
<EXHAUSTED_CODING_FAILURES_JSON>

Produce a compressed planning memory, not a transcript. Return JSON with exactly three keys: Question,
Finding, and Errors appear in coding. Each Question entry must start with its Q-id, and each Finding
entry with its evidence ID and originating Q-id. Cover every supplied Q-id and every validated evidence
```

```
ID exactly once. Preserve qualitative conclusions, necessary comparison conditions, limitations, and
any applicable agreement restrictions. [value omitted] and [quantity omitted] in the supplied insights
are redactions, not text to copy into memory. Finding must omit measured result values in digits or
words: counts, percentages, averages, distances, durations, sample sizes, and ranges. Original evidence
keeps them. Categorical labels such as day_1, Day 1, and route_2 may remain. A brief count of compared
categories is allowed only when essential to describe the finding's scope. Retain a numeric question
condition only if needed to distinguish its objective. Use one terse entry per ID: a short analytical
objective for Question, and the qualitative result plus indispensable context for Finding. Aim for at
most 260 characters per Question, 450 per Finding, and about 2,000 overall; hard limits are 400 per
Question, 900 per Finding, and 4,000 total. Do not copy source paragraphs, statistics, raw code, or
repeated goal/profile details. Do not add unverified findings. For each supplied exhausted coding
failure, keep one concise, actionable entry beginning with its Q-id; these entries guide the follow-up
code writer, not question planning. Use [] only when no exhausted coding failures were supplied. Do not
propose next questions.
{"Question":["Q1: ..."],"Finding":["E1: Q1 - ..."],"Errors appear in coding":[]}
```

**Prompt 11: Follow-up question planning**

```
SYSTEM:
Propose three next analysis questions using the supplied goal, memory, and data profile. Prioritize
gaps in what Finding actually established. Do not write code, answer questions, or infer new field
relations. Return JSON only.

USER:
HIGH-LEVEL GOAL:
<GOAL>

COMPRESSED FIRST-ROUND MEMORY:
<PLANNING_MEMORY_JSON>

DATA PROFILE:
<DATA_PROFILE_JSON>

Generate exactly three goal-relevant next questions. Use memory.Question to see
what was attempted and memory.Finding to see what was actually established.
First identify which parts of the high-level goal remain unanswered or only
partly answered by the Findings. Prefer questions that fill those gaps and can
produce a new main insight for the goal. A Question alone does not mean its
objective was answered. If a Finding is partial, ask for the specific missing
comparison or pattern; do not rerun the same attempted analysis unchanged.
Avoid near-rephrasings and superficial variants that change only wording, a
minor subgroup, time window, or metric without a meaningful new analytical angle.
Do not spend a follow-up on a peripheral data property while a substantive
part of the goal remains unanswered. If the Findings already answer the goal,
explore useful new goal-relevant directions. Findings are verified context,
not a requirement that every next question refine them.
Use the data profile to ensure the questions are answerable.
Prefer useful, distinct descriptive or comparative directions. A question may connect
findings or share a topic when it seeks a clearly different outcome, comparison,
or pattern that serves the goal. The three candidates should also differ in their
analytical objectives, not merely their phrasing or small slices of the same analysis.
Do not default to explanation, interaction effects, controls, robustness checks,
or more complex methods merely because findings
exist. Do not assume a hypothesized relationship is true or an unverified join works.
Respect the conditions and conflict-use restrictions retained in Finding.

Each question must pursue one clear analytical objective with a single question mark.
Related metrics may be combined within one coherent comparison; unrelated objectives
must not be bundled. Plan its base analysis unit and optional aggregation unit.
```

```
For each candidate, state in expected_new_information the specific unanswered
or partial aspect it would resolve and the new conclusion it could support.
Do not presume the result or copy an answer already present in Finding.
Return exactly three candidates. related_evidence_ids may contain only evidence IDs
listed in Finding, and only when the new analysis genuinely relies on those findings;
otherwise use []. relation_ids may contain only IDs in the data profile.

Return JSON only:
{"candidates":[
{"question":"...?","expected_new_information":"...","related_evidence_ids":[],"relation_ids":[],
"analysis_granularity":{"base":"trajectory/trip","aggregation":null}},
{"question":"...?","expected_new_information":"...","related_evidence_ids":[],"relation_ids":[],
"analysis_granularity":{"base":"group","aggregation":"..."}},
{"question":"...?","expected_new_information":"...","related_evidence_ids":[],"relation_ids":[],
"analysis_granularity":{"base":"trajectory/trip","aggregation":null}}
]}

DATA CONTEXT:
<DATA_CONTEXT>
```

**Prompt 12: Follow-up novelty and goal coverage**

```
SYSTEM:
Review proposed questions against the goal and compressed memory. Flag repetitions and clear failures
to address unanswered goal content. Return JSON only.

USER:
HIGH-LEVEL GOAL:
<GOAL>

COMPRESSED FIRST-ROUND MEMORY:
<PLANNING_MEMORY_JSON>

DATA PROFILE:
<DATA_PROFILE_JSON>

PROPOSED FOLLOW-UP QUESTIONS:
<CANDIDATE_QUESTIONS_JSON>

Decide whether a proposed question substantially repeats an objective in
memory.Question or asks for an answer already contained in memory.Finding.
Flag near-rephrasings and superficial changes of subgroup, time window, or
metric. For a match to a Finding, report its originating Q-id as
initial_question_id. A shared topic or field alone is not enough: allow a
specific missing comparison or pattern when the original Finding is partial.

Also flag a coverage_issue when a candidate's expected new information is
already established by a Finding, does not directly advance the high-level
goal, or examines a peripheral data property while an obvious substantive
part of the goal remains unanswered. Do not require a predetermined list of
relations or a match to benchmark answers. If the Findings already address
the goal, allow useful new goal-relevant exploration. Do not judge statistical
correctness or answerability here.
Return {"duplicates":[{"follow_up_id":"F1","initial_question_id":"Q1","reason":"..."}],
"coverage_issues":[{"follow_up_id":"F2","reason":"..."}]};
use empty lists when none apply.

DATA CONTEXT:
<DATA_CONTEXT>
```

**Prompt 13: Observation export for conditional refinement**

```
SYSTEM:
Export computed observational evidence; Python only.

USER:

Create a small supplemental computation for the FIXED QUESTION using full named inputs.
This is a research prototype: export at most THREE relevant numerical measures with
one or two categorical conditioning dimensions. Do not explore adjacent questions.
Use the old code as a field/metric guide, but correct a wrong or insufficient statistic
when the question requires a different relation. Never reuse a conclusion as data.
Useful measures: per-entity/trip/day arrival or duration; count per complete time bin;
distance/speed; or 0/1 indicators for a precisely computed event or sequence relation.
For nearest-next-destination questions compute actual next-stop vs available candidates;
sequence similarity alone cannot answer that question. Do not use linear mean/SD of
angles, clock times crossing midnight, or raw lat/lon as a surrogate for route shape.
If this numeric conditional grammar does not fit, export no measures and explain why.

API (these exact classes already exist):
from trajstoryagent_v1.analysis_tools import AnalysisSession
from trajstoryagent_v2.math_refinement import EvidenceCollector
session = AnalysisSession.from_environment()
df = session.load_table('main')  # use ONLY named file IDs from input catalogue
collector = EvidenceCollector()
collector.add_measure(frame, value='numeric_column', conditions=['group_column'],
    observation_id=['entity_id', 'date'], unit='minutes', observation_unit='entity-day',
    description='arrival time', scope='observed weekdays, all specified users',
    kind='continuous')  # or kind='indicator' for EXACT 0/1 values
collector.finish()  # required; or finish(reason='unsupported relation ...') if no measures

The frame contains one record per stated observational unit, not duplicated GPS points
masquerading as trips/users. observation_id columns must uniquely identify every row.
Condition columns are complete, meaningful categorical partitions (<=12 values each).
Use existing meaningful bins; if needed state all bin boundaries explicitly. Never
select top groups, arbitrary IDs, or most favorable subsets to satisfy the size cap.
Preserve all observations within the stated question scope. Include zeros for complete
observed intervals where appropriate; do not invent zero activity outside observation.
Numerical values MUST be computed, not typed as answers. Null handling is explicit.
Validate join cardinality. Source tables must never be modified. No network, os, sys,
pathlib, direct file reads/writes, plots, session.emit_result(), prints, or narrative.
The only artifact writer is collector.finish(). pandas/numpy/math/scipy/sklearn and
the two imported project APIs are available. Return Python source only.

MODEL-FACING INPUTS:
{"goal": <GOAL>, "context": <DATA_CONTEXT>, "files": <NAMED_INPUT_FILES>,
 "question": <QUESTION>, "granularity": <GRANULARITY>, "profile": <PROFILE>}

<ORIGINAL_COMPUTATION_CODE>
```

**Prompt 14: Interpretation after conditional refinement**

```
SYSTEM:
Interpret computed evidence only. JSON only.

USER:
Answer the fixed analytical question using the original computed evidence and the new deterministic
certificate facts. Stay within the original question. Preserve important conditions, comparisons,
numerical support, within-group variation and material exceptions. Do not describe algorithms or
certificate IDs in the insight. A pooled fit does not prove groups are identical. Indicator means are
empirical proportions. Do not infer statistical significance, causality or population-level
generalizations. A compression default is approximate; use supplied exact means/ranges when precision
matters. If the new measure is irrelevant or has a wrong analytical unit, set use_refinement=false and
explain. Return JSON {"use_refinement":true,"insight":"one coherent factual insight, at most 180
words","used_fact_ids":["C1"],"limitations":["..."],"binding_concerns":[]}. List only actually used
facts; do not invent evidence.
{"goal": <GOAL>, "question": <QUESTION>, "granularity": <GRANULARITY>,
 "computed_original": <STATISTICS_TABLES_SCOPE_LIMITATIONS>,
 "certificate_facts": <FACT_ID_TEXT_LIST>}
```

**Prompt 15: Final summary generation**

```
SYSTEM:
You are TrajStory A5. Summarize finalized branch insights. Return JSON only.

USER:
HIGH-LEVEL GOAL:
<GOAL>

FINAL INSIGHTS (both rounds have completed conditional decomposition and compression):
<FINAL_INSIGHTS_JSON>

EVIDENCE CONDITIONS AND LIMITATIONS:
<EVIDENCE_CONDITIONS_JSON>

Generate a coherent, goal-oriented summary based only on the supplied final insights.
Organize related findings, preserve conditions, exceptions, conflicts and uncertainty.
Do not add facts, numbers or causal interpretations absent from these insights.
Respect agreement_context.use_rules; deferred hypotheses are not established findings.
Do not generate, rewrite, merge or replace the insights list. It is fixed by the branches.
Return exactly {"summary":"..."}.
```

### *B. Analytical Operation Counting*

The counting prompt below applies the current common granularity to each evaluation unit. The caller supplies the target and available input schema separately and counts the returned operations. Its response schema requires `task_id`, `status` (`OK` or `UNSCORABLE`), `operations` (each with `operator` and `description`), `assumptions`, and `missing_context`. Task programs share identical necessary operations, whereas each standalone insight starts from the supplied inputs.

**Prompt 16: Analytical operation counting**

```
# Analytical operation counting

Estimate the shortest conventional analytical program that produces the requested findings. Annotate
operations, not exact answer reproduction. Use the same granularity across benchmarks. Do not aim for a
particular count.

The goal/question and reference insights define the analytical targets. Evidence, descriptions and
insight_value clarify them; they are reference outputs, never supplied input data. Do not turn
incidental background counts, explanatory diagnostics or every supporting number into independent
requirements. Explicitly requested statistics must be covered. At task level, cover all requested
insights in one program and share identical necessary operations across insights. Share only operations
with the same inputs, subset, measure and analytical stage; do not merge different measures or
different comparisons merely because they use the same data. At standalone insight level, start from
the supplied inputs and cover only that insight. Task counts come from the shared task program, not the
sum of standalone insight counts.

Operators:
- SELECT: retrieve existing facts needed as final outputs. Reading inputs, key matching and
option-letter formatting are free.
- FILTER: form a necessary subset. Combine all conditions on the same subset into one FILTER. Separate
subsets with different semantic roles or a later filter dependent on a computed value remain separate.
- ORGANIZE: necessary grouping or chronological ordering. Multi-key grouping is one operation.
Formatting and display sorting are free.
- COMPUTE: a conventional derived measure, statistic, prediction or classification. Do not expand its
internal arithmetic, model training or implementation. One COMPUTE may calculate only ONE measure type
at ONE analytical stage. The same measure across multiple objects, groups or files counts once within
the evaluated program. Different measure types MUST be separate operations: route share, route-length
ratio and route dispersion are three COMPUTEs; trip count, distinct-driver count and request-source
share are three COMPUTEs. Dependent analytical levels remain separate. Never use a bundled step such as
calculate all metrics or summarize all measures. Use fields actually supplied in data_files directly;
do not treat fields mentioned only in evidence as existing input columns.
- COMPARE: an explicit comparison, ranking, extremum or trend judgment. One COMPARE may compare only
ONE measure type. Comparing that same measure across multiple groups counts once within the evaluated
program; comparing driver-pool size, time distribution and request-source share requires three separate
COMPARE operations. When the target requires comparison, this operation is required even if group
statistics have already been computed. Do not add sorting for the same judgment.

Binding counting precedents:
1. Predicting the next location from historical observations is one COMPUTE even if the prediction
model is unspecified. Record a concise assumption instead of refusing.
2. For speed anomalies from timestamped coordinates with no speed or specified threshold, consistently
use COMPUTE speed, COMPUTE a data-driven threshold (use the conventional 1.5-IQR rule), FILTER
anomalies: three operations. Do not switch to an assumed constant threshold to reduce the count. If
speed or a threshold is actually supplied, omit its derivation; honor any method specified by the task.
3. Given date and trip distance, comparing weekday/weekend mean distance is ORGANIZE groups, COMPUTE
group means, COMPARE: three operations. Calendar categorization is part of this grouping.
4. Finding anomalies does not require an additional count. Count them only when their number is an
explicit analytical target. Incidental collection size is not mandatory.
5. Filtering Monday morning urban taxi records and computing mean speed is FILTER all conditions,
COMPUTE mean: two operations when these fields are supplied.
6. Calculating each trajectory's length from its points is one COMPUTE, including adjacent distances
and their sum. Computing group mean trajectory length is another COMPUTE. Thus trajectory length,
weekday/weekend grouping, group means, comparison is four operations. Do not split trajectory length
into low-level arithmetic, or merge it with the group mean.
7. Predict destination then retrieve nearby POIs is COMPUTE followed by SELECT when the mapping is
available.

Exclude loading, joins/key matching, formatting, plotting, prose generation and recommendations.
```

```
Missing exact values, thresholds, parameters or algorithm names do not make the operation structure
unscorable: use the fixed precedents or a conventional method and state up to three count-affecting
assumptions. Do not invent actual input facts or essential external mappings. UNSCORABLE is reserved
for an uninterpretable target or genuinely missing essential source/meaning that prevents a defensible
structural program. Purely nonanalytical prose may have an empty OK program.

Return concise operations, not chain-of-thought. OK requires empty missing_context; UNSCORABLE requires
empty operations and a specific missing_context. The caller counts the operations list.
```

### *C. Benchmark Quality Assessment*

The following prompts are taken from the latest saved assessment inputs. Both use GPT-6 Luna with temperature 0 and reasoning disabled in two independent rounds. Case material contains the goal, reference insights, and available source profiles and samples; images, when available, are supplied separately. HeurekaBench uses a distinct evidence description because its profiles expose structural H5AD metadata rather than expression values. These prompts evaluate dataset quality, rather than agent outputs.

**Prompt 17: Benchmark quality assessment**

```
Assess this DATASET CASE, not any agent's generated answer. Follow the
four UniDataBench Appendix B.1 dimensions using a 1-5 integer scale:
1-2 poor, 3 acceptable, 4-5 excellent.

Data suitability: Do the actual tables have the fields, coverage, granularity,
quality and coherent schema needed for the analytical goal and reference insights?
Relevance: Does EACH reference insight directly address the stated goal? Mark
any off-goal insight by its 1-based index.
Consistency: Consider ALL reference insights together. Flag contradictions,
including claims that conflict across tables or figures.
Rationality: Are the claims plausible and supported by supplied data and figures?
Do not treat a gold answer's asserted value, plot description, or code snippet
as proof that it is correct. If the supplied profile/sample/plots cannot prove
a number, state that limitation and avoid a perfect score on that basis alone.

The CSV profile is computed from the source files; displayed rows are examples,
not the whole dataset. Images are numbered and mapped to insight indices.
Return JSON ONLY with exactly these keys:
data_suitability, relevance, consistency, rationality, flagged_insights,
limitations. Each dimension is an object {"score": integer 1..5,
"reason": short concrete string}. flagged_insights is a list of objects
{"index": integer, "issue": short string}; limitations is a list of strings.
Keep reasons concise and cite concrete fields, insight indices or images.

CASE MATERIAL:
<CASE_MATERIAL_JSON>
```

**Prompt 18: HeurekaBench quality assessment**

```
Assess this DATASET CASE, not any agent's generated answer. Follow the
four UniDataBench Appendix B.1 dimensions using a 1-5 integer scale:
1-2 poor, 3 acceptable, 4-5 excellent.

Data suitability: Do the actual tables have the fields, coverage, granularity,
quality and coherent schema needed for the analytical goal and reference insights?
Relevance: Does EACH reference insight directly address the stated goal? Mark
any off-goal insight by its 1-based index.
Consistency: Consider ALL reference insights together. Flag contradictions,
```

```
including claims that conflict across tables or figures.
Rationality: Are the claims plausible and supported by supplied data and figures?
Do not treat a gold answer's asserted value, plot description, or code snippet
as proof that it is correct. If the supplied profile/sample/plots cannot prove
a number, state that limitation and avoid a perfect score on that basis alone.

H5AD profiles below are read from actual source files but inspect only structural metadata (shapes,
fields, and limited categories), not expression values. Ancillary CSV/text previews are samples. No
reference figure or analysis code is supplied here. Do not infer that a gold answer was reproduced.
Return JSON ONLY with exactly these keys:
data_suitability, relevance, consistency, rationality, flagged_insights,
limitations. Each dimension is an object {"score": integer 1..5,
"reason": short concrete string}. flagged_insights is a list of objects
{"index": integer, "issue": short string}; limitations is a list of strings.
Keep reasons concise and cite concrete fields, insight indices or images.

CASE MATERIAL:
<CASE_MATERIAL_JSON>
```

## REFERENCES


[1] Y. Zhu, J. J. Yu, X. Zhao, X. Zhou, L. Han, X. Wei, and Y. Liang, "Unitraj: learning a universal trajectory foundation model from billion-scale worldwide traces," in *Proceedings of the 39th International Conference on Neural Information Processing Systems*, ser. NIPS '25. Red Hook, NY, USA: Curran Associates Inc., 2026. [Online]. Available: https://doi.org/10.52202/085713-1407

[2] K. V. Olesen, A. N. Christensen, and L. K. H. Clemmensen, "AIS trajectories from danish waters for abnormal behavior detection," 2023.

[3] L. Moreira-Matias, M. Ferreira, J. Mendes-Moreira *et al.*, "Taxi service trajectory - prediction challenge, ECML PKDD 2015," 2013. [Online]. Available: https://doi.org/10.24432/C55W25

[4] L. Wu, H. Wen, H. Hu, X. Mao, Y. Xia, E. Shan, J. Zheng, J. Lou, Y. Liang, L. Yang, R. Zimmermann, Y. Lin, and H. Wan, "LaDe: The first comprehensive last-mile express dataset from industry," in *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, 2024, pp. 5991–6002.

[5] Ville de Montréal, "Déplacements mtl trajet," Dataset, 2017, CC-BY 4.0. [Online]. Available: https://www.donneesquebec.ca/recherche/dataset/vmtl-mtl-trajet

[6] A. Lima, R. Stanojevic, D. Papagiannaki, P. Rodriguez, and M. C. González, "Understanding individual routing behaviour," *Journal of the Royal Society Interface*, vol. 13, no. 116, p. 20160021, 2016.

[7] Federal Highway Administration, "Multiday gps travel behavior data for travel analysis," U.S. Department of Transportation, Federal Highway Administration, Tech. Rep. FHWA-HEP-15-026, 2015. [Online]. Available: https://rosap.ntl.bts.gov/view/dot/55893

[8] B. K. T. Edjah and S. Li, "Operational and spatial determinants of pickup performance in last-mile delivery: Evidence from the lade jilin dataset," *Research in Transportation Business & Management*, vol. 68, p. 101735, 2026.

[9] R. Mahes, M. A. A. Boon, A. Kuiper, M. Mandjes, and F. Bijma, "Dynamic time-window updates in last-mile delivery: 'follow your parcel'," 2024, sSRN Working Paper No. 4919725. [Online]. Available: https://ssrn.com/abstract=4919725

[10] L. Moreira-Matias, J. Gama, M. Ferreira, J. Mendes-Moreira, and L. Damas, "Predicting taxi-passenger demand using streaming data," *IEEE Transactions on Intelligent Transportation Systems*, vol. 14, no. 3, pp. 1393–1402, 2013.

[11] J. J. Tang, "Individual heterogeneity and cultural attitudes in credence goods provision," *European Economic Review*, vol. 126, p. 103442, 2020.

[12] A. Hörteborn and J. W. Ringsberg, "A method for risk analysis of ship collisions with stationary infrastructure using ais data and a ship manoeuvring simulator," *Ocean Engineering*, vol. 235, p. 109396, 2021.

[13] J. Borg, A. Kiiski, E. Luoma, M. Hosseini, T. H. Iversen, H. Haapasaari, S. Burman, H. Gluver, and A. Lentz, "BRISK II traffic analysis," BRISK II Project, Tech. Rep. D2.3, 2025, deliverable D2.3. [Online]. Available: https://civil-protection-knowledge-network.europa.eu/system/files/2026-01/brisk-ii-d2.3-traffic-analysisi_submitted-30.9.2025_1.pdf

[14] T. Keel, "Can we predict why people travel within a city?: A study analysing the spatial and temporal characteristics of travel intention within montréal, canada." Master's thesis, 08 2019. [Online]. Available: https://www.researchgate.net/publication/344072488

[15] O. d. J. L. Torres, "Decision making for urban mobility: A macro, meso and micro analysis," Master's thesis, Concordia University, 2020. [Online]. Available: https://spectrum.library.concordia.ca/id/eprint/986456/

[16] OpenAI, "GPT-6 Luna," https://developers.openai.com/api/docs/models/gpt-6-luna, accessed: 2026-10-18.

[17] H. Weng, Z. Liu, Y. Song, X. Yin, X. Chen, and W. Zhang, "UniDataBench: Evaluating data analytics agents across structured and unstructured data," in *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, M. Liakata, V. P. Moreira, J. Zhang, and D. Jurgens, Eds. San Diego, California, United States: Association for Computational Linguistics, Jul. 2026, pp. 33 755–33 780. [Online]. Available: https://aclanthology.org/2026.acl-long.1556/

[18] Qwen Team, "Qwen3.8-Flash," Aug. 2026, qwenCloud. [Online]. Available: https://www.qwencloud.com/models/qwen3.8-flash

[19] W. Zhang, L. Zeng, Y. Xiao, Y. Li, C. Cui, Y. Zhao, R. Hu, Y. Liu, Y. Zhou, and B. An, "Agentorchestra: Orchestrating multi-agent intelligence with the tool-environment-agent(tea) protocol," 2026. [Online]. Available: https://arxiv.org/abs/2506.12508

[20] G. Sahu, A. Puri, J. A. Rodriguez, A. Abaskohi, M. Chegini, A. Drouin, P. Taslakian, V. Zantedeschi, A. Lacoste, D. Vazquez, N. Chapados, C. Pal, S. Rajeswar, and I. H. Laradji, "InsightBench: Evaluating Business Analytics Agents Through Multi-Step Insight Generation," in *International Conference on Learning Representations*, 2025. [Online]. Available: https://mlanthology.org/iclr/2025/sahu2025iclr-insightbench/

[21] B. P. Majumder, H. Surana, D. Agarwal, B. D. Mishra, A. Meena, A. Prakhar, T. Vora, T. Khot, A. Sabharwal, and P. Clark, "DiscoveryBench: Towards Data-Driven Discovery with Large Language Models," in *International Conference on Learning Representations*, 2025. [Online]. Available: https://mlanthology.org/iclr/2025/majumder2025iclr-discoverybench/

[22] LangChain, "Pandas DataFrame Agent," https://github.com/langchain-ai/langchain-experimental/blob/main/libs/experimental/langchain_experimental/agents/agent_toolkits/pandas/base.py, accessed: 2026-10-17.

[23] M. Sun, Y. Xie, Y. Wu, R. Han, B. Jiang, D. Sun, Y. Yuan, and J. Huang, "Dsaeval: Evaluating data science agents on a wide range of real-world data science problems," 2026. [Online]. Available: https://arxiv.org/abs/2601.13591